Clase Práctica Sorting

2C2026
Turno Noche
Belén Alvariñas

Pequeño repaso

Algoritmo	¿Complejidad en peor caso?😯	Estable
Insertion Sort		
Selection Sort		
Quick Sort		
Merge Sort		
Heap Sort		

Pequeño repaso

Algoritmo	¿Complejidad en peor caso?😯	Estable
Insertion Sort	$O(n^2)$	
Selection Sort		
Quick Sort		
Merge Sort		
Heap Sort		

Pequeño repaso

Algoritmo	¿Complejidad en peor caso?😯	Estable
Insertion Sort	$O(n^2)$	
Selection Sort	$O(n^2)$	
Quick Sort		
Merge Sort		
Heap Sort		

Pequeño repaso

Algoritmo	¿Complejidad en peor caso?😯	Estable
Insertion Sort	$O(n^2)$	
Selection Sort	$O(n^2)$	
Quick Sort	$O(n^2)$	
Merge Sort		
Heap Sort		

Pequeño repaso

Algoritmo	¿Complejidad en peor caso?😯	Estable
Insertion Sort	$O(n^2)$	
Selection Sort	$O(n^2)$	
Quick Sort	$O(n^2)$	
Merge Sort	$O(n.\log n)$	
Heap Sort		

Pequeño repaso

Algoritmo	Complejidad en peor caso	¿Estable?😯
Insertion Sort	$O(n^2)$	
Selection Sort	$O(n^2)$	
Quick Sort	$O(n^2)$	
Merge Sort	$O(n.\log n)$	
Heap Sort	$O(n.\log n)$	

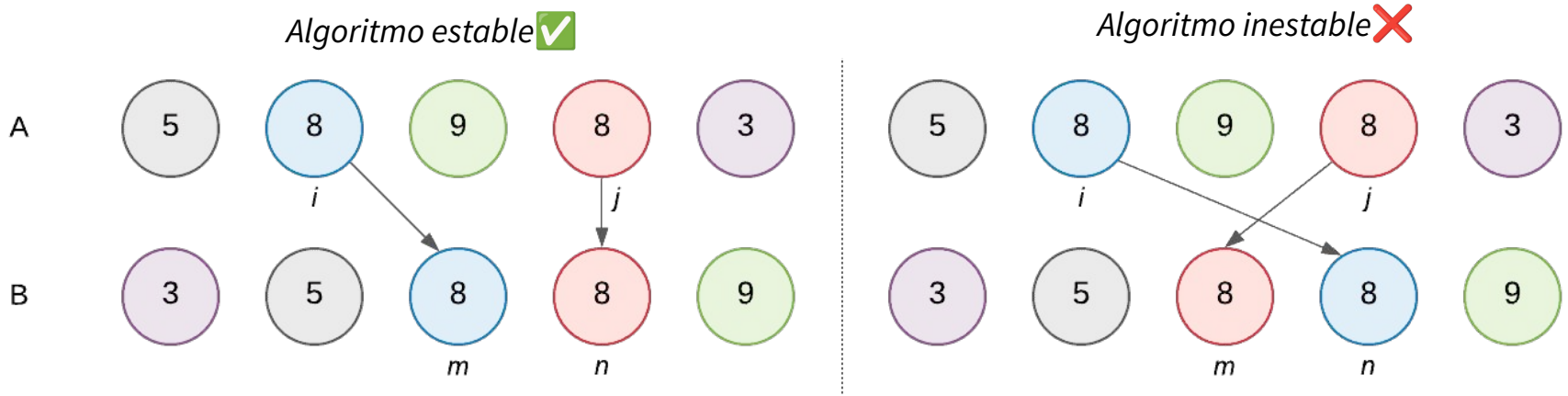
Pequeño repaso

¿Qué significa que un algoritmo sea estable?

Pequeño repaso

¿Qué significa que un algoritmo sea estable?

Un algoritmo de ordenamiento es estable si dos registros i y j con claves iguales mantienen su orden relativo luego del ordenamiento.



Pequeño repaso

Algoritmo	**Complejidad en peor caso**	**¿Estable?😲**
Insertion Sort	$O(n^2)$	
Selection Sort	$O(n^2)$	
Quick Sort	$O(n^2)$	
Merge Sort	O(n.log n)	
Heap Sort	O(n.log n)	

Pequeño repaso

Algoritmo	**Complejidad en peor caso**	**¿Estable?😲**
Insertion Sort	$O(n^2)$	Sí
Selection Sort	$O(n^2)$	
Quick Sort	$O(n^2)$	
Merge Sort	O(n.log n)	
Heap Sort	O(n.log n)	

Pequeño repaso

Algoritmo	**Complejidad en peor caso**	**¿Estable?😲**
Insertion Sort	$O(n^2)$	Sí
Selection Sort	$O(n^2)$	No
Quick Sort	$O(n^2)$	
Merge Sort	O(n.log n)	
Heap Sort	O(n.log n)	

Pequeño repaso

Algoritmo	**Complejidad en peor caso**	**¿Estable?😲**
Insertion Sort	O(n²)	Sí
Selection Sort	O(n²)	No
Quick Sort	O(n²)	No
Merge Sort	O(n.log n)	
Heap Sort	O(n.log n)	

Pequeño repaso

Algoritmo	**Complejidad en peor caso**	**¿Estable?😲**
Insertion Sort	$O(n^2)$	Sí
Selection Sort	$O(n^2)$	No
Quick Sort	$O(n^2)$	No
Merge Sort	O(n.log n)	Sí
Heap Sort	O(n.log n)	

Pequeño repaso

Algoritmo	Complejidad en peor caso	Estable
Insertion Sort	$O(n^2)$	Sí
Selection Sort	$O(n^2)$	No
Quick Sort	$O(n^2)$	No
Merge Sort	$O(n.\log n)$	Sí
Heap Sort	$O(n.\log n)$	No

Sobre los algoritmos

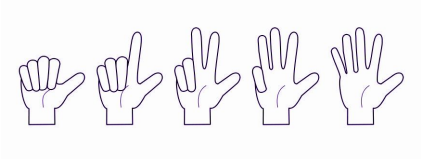
Estos algoritmos ordenan arreglos comparando los elementos, es decir, son algoritmos de sorting por comparación.

Vieron en la teórica que en el peor caso son $\Omega(n \log n)$.

¿Puede haber algo mejor?

Sí😁, por ejemplo, si tenemos **información** de los elementos.

Counting Sort (Naturales)



¿Qué información tenemos?

Asume que los elementos de un arreglo A de naturales son **menores** que un cierto valor **max**.

Idea:

1) Construir un arreglo B de tamaño k y guardar en la posición i, la cantidad de apariciones de i en A.
2) Reconstruir el arreglo A, poniendo tantas copias de cada valor $j \in \{0..k\}$, como lo indique B[j].

Counting Sort (Naturales): Ejemplo

A =

7	1	4	1	4	2	12	1

max = 12

Counting Sort (Naturales): Ejemplo

A =

7	1	4	1	4	2	12	1

max = 12

Paso 1 :

B =

0	3	1	0	2	0	0	1	0	0	0	0	1

Counting Sort (Naturales): Ejemplo

A =

7	1	4	1	4	2	12	1

max = 12

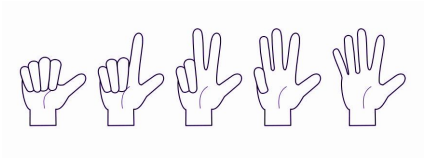
Paso 1 : **B =**

0	3	1	0	2	0	0	1	0	0	0	0	1

Paso 2 : **output =**

1	1	1	2	4	4	7	12

Counting Sort (Naturales):



```
proc CountingSortNat(A: Array<int>, max: int)
    // B guarda la cantidad de apariciones de cada valor.
    intervaloSize = max + 1

    B := new Array<int>[intervaloSize]
    for (i = 0, i < length(B)-1, i+1)
        B[i] := 0
    endfor

    // Cuento la cantidad de apariciones de cada elemento.
    for (i = 0, i < length(A), i+1)
        B[A[i]] := B[A[i]] + 1
    endfor

    // Inserto en A cada elemento tantas veces como
    // aparece.
    indexA := 0
    for (i = 0, i < length(B)-1, i+1)
        for (j = 0, j < B[i], j+1)
            A[indexA] := i
            indexA := indexA + 1
        endfor
    endfor
endproc
```

Counting Sort para Naturales

¿Complejidades?

Complejidad : O(n + max)

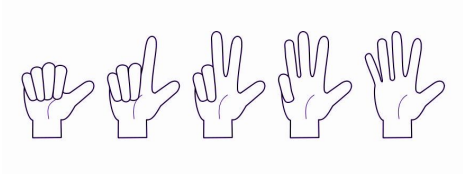
```
proc CountingSortNat(A: Array<int>, max: int)
    // B guarda la cantidad de apariciones de cada valor.
    intervaloSize = max + 1

    B := new Array<int>[intervaloSize]
    for (i = 0, i < length(B)-1, i+1)
        B[i] := 0
    endfor

    // Cuento la cantidad de apariciones de cada elemento.
    for (i = 0, i < length(A), i+1)
        B[A[i]] := B[A[i]] + 1
    endfor

    // Inserto en A cada elemento tantas veces como
    // aparece.
    indexA := 0
    for (i = 0, i < length(B)-1, i+1)
        for (j = 0, j < B[i], j+1)
            A[indexA] := i
            indexA := indexA + 1
        endfor
    endfor
endproc
```

Counting Sort



Misma idea que CountingSortNat, pero esta versión se puede usar para números **enteros** en un rango entre **min** y **max**.

Complejidad : O(max - min + n)

```
proc CountingSort(A: Array<int>, min: int, max: int)
    intervaloSize := max - min + 1

    B := new Array<int>[intervaloSize]
    for (i=0, i < length(B)-1, i+1)
        B[i] := 0
    endfor

    // Cuento la cantidad de apariciones de cada elemento.
    for (i=0, i < length(A), i+1)
        B[A[i] - min] := B[A[i] - min] + 1
    endfor

    // Inserto en A cada elemento tantas veces como
    //aparece.
    indexA := 0
    for (i=0, i < length(B)-1, i+1)
        for (j=0, j < B[i], j+1)
            A[indexA] := i + min
            indexA := indexA + 1
        endfor
    endfor
endproc
```

Bucket Sort 

¿Qué información tenemos?

Que los elementos pueden separarse (según algún criterio) en **M** categorías ordenables. Es decir, para $i < j$, todo elemento de la categoría i es menor que toda elemento de la categoría j.

Idea:

1) Construir un arreglo de M listas y guardar los elementos de la categoría i en la i-ésima lista con la función de asignación h.
2) Reconstruir el arreglo A, concatenando las listas en orden.

Bucket Sort : Ejemplo

A =								
	(“Román”, TN)	(“Camilo”, TT)	(“Dani”, TT)	(“Elias”, TN)	(“Dami”, TM)	(“Nico”, TN)	(“Anto”, TN)	(“Diego”, TM)

Me piden ordenar a los profes de la materia según su turno siguiendo como orden: TM, TT, TN

input:

- A
- M = 3 (separamos en 3 categorías: TM, TT, TN)

¿Cómo sabemos a qué bucket va Nico que es del TN? Eso lo define nuestra función h (función de asignación) que definimos con palabras:

- h: si el turno es TM, le corresponde el bucket 0, TT bucket 1 y TN bucket 2

  👀OJO!! Esta función tiene una complejidad asociada como todas los pasos del algoritmo y tenemos que tenerla en cuenta a la hora de resolver problemas 👀

Bucket Sort : Ejemplo

M= 3, h

input:	(“Román” , TN)	(“Camilo” , TT)	(“Dani”, TT)	(“Elias”, TN)	(“Dami”, TM)	(“Nico”, TN)	(“Anto”, TN)	(“Diego”, TM)

Paso 1:

(“Dami”, TM)	(“Diego”, TM)



0

(“Camilo” , TT)	(“Dani”, TT)



1

(“Román” , TN)	(“Elias”, TN)	(“Nico”, TN)	(“Anto”, TN)



2

Bucket Sort : Ejemplo

Paso 1:

(“Dami”, TM)	(“Diego”, TM)



0

(“Camilo”, TT)	(“Dani”, TT)



1

(“Román”, TN)	(“Elias”, TN)	(“Nico”, TN)	(“Anto”, TN)



2

Paso 2:

output:

(“Dami”, TM)	(“Diego”, TM)	(“Camilo”, TT)	(“Dani”, TT)	(“Román”, TN)	(“Elias”, TN)	(“Nico”, TN)	(“Anto”, TN)

Bucket Sort



```
proc BucketSort(A: Array<T>, M: int, h)

    n := length(A)
    B := new Array<lista> [ M ]

    for (i=0, i < n, i+1)
        Insertar A[i] en la lista B[h(A[i])]
    endfor

    Concatenar B[0], B[1], ..., B[M-1]
endproc
```

Bucket Sort



¿Complejidades?

```
proc BucketSort(A: Array<T>, M: int, h)

    n := length(A)
    B := new Array<lista> [ M ]

    for (i=0, i < n, i+1)
        Insertar A[i] en la lista B[h(A[i])]
    endfor

    Concatenar B[0], B[1], ..., B[M–1]
endproc
```

Complejidad : O (n * costoDe Asignar + M)

Radix Sort (versión naturales)

¿Qué información tenemos?

Que son números naturales, en una base *b* y que se pueden descomponer en sus cifras significativas.

Idea:

1) Ordenar los valores mirando solo el dígito 1
2) Ordenar los valores mirando solo el dígito 2 (manteniendo el orden en caso de empate)
3) Ordenar los valores mirando solo el dígito 3 (manteniendo el orden en caso de empate)
4) …

Radix Sort (versión naturales) : Ejemplo

input:

325	19	426	17	445	123	175

Radix Sort (versión naturales) : Ejemplo

input:

325	19	426	17	445	123	175

12**3**
32**5**
44**5**
17**5** ⟶
42**6**
1**7**
1**9**

paso 1

Radix Sort (versión naturales) : Ejemplo

input:

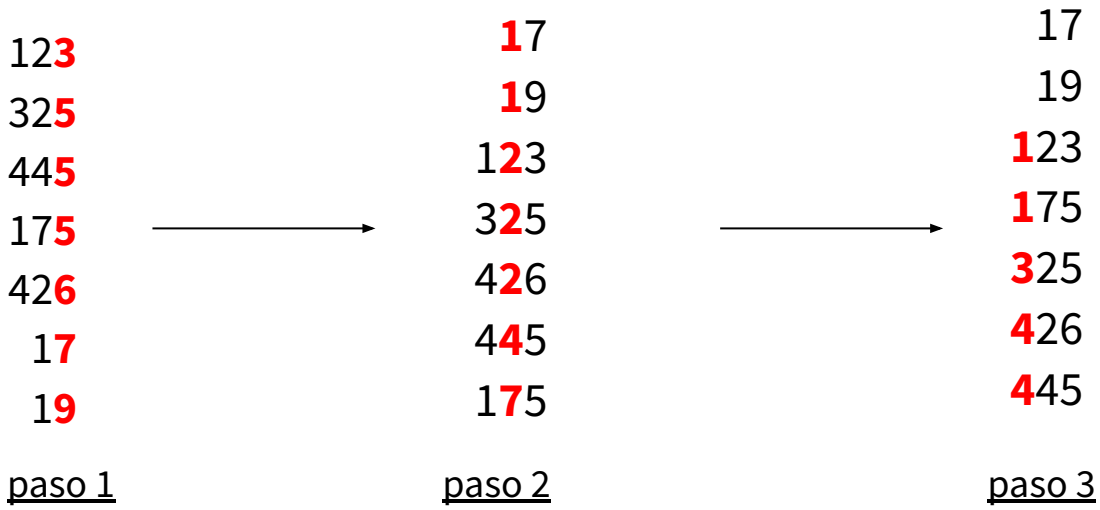
325	19	426	17	445	123	175



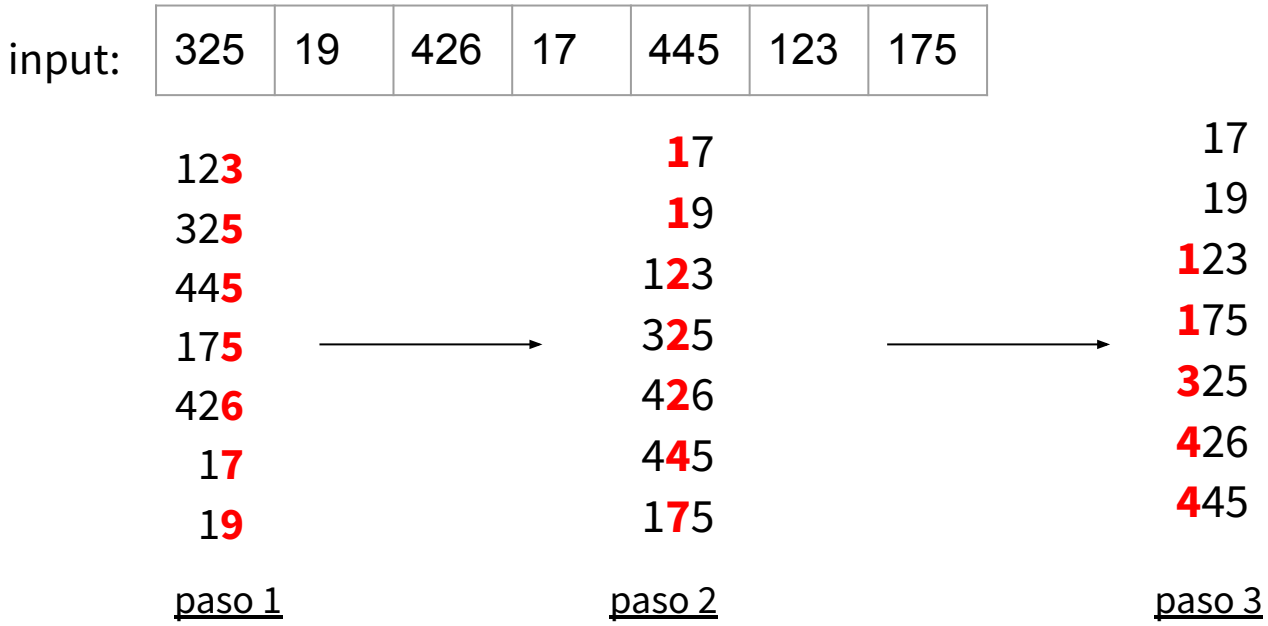
Radix Sort (versión naturales) : Ejemplo

input:

325	19	426	17	445	123	175



Radix Sort (versión naturales) : Ejemplo



Radix Sort (versión naturales)

```
proc RadixSort(A: Array<int>, d: int)

    for (i=0, i <= d, i+1)
        // Del menos significativo al más significativo.
        Ordenar el arreglo A según el dígito i,
        de forma estable
    endfor
endproc
```

Llamamos *d* a la cantidad máxima de dígitos de los elementos y suponemos que el dígito 1 es el menos significativo de cada valor

¿Complejidades?

Radix Sort (versión naturales)

```
proc RadixSort(A: Array<int>, d: int)

    for (i=0, i <= d, i+1)
        // Del menos significativo al más significativo.
        Ordenar el arreglo A según el dígito i,
        de forma estable
    endfor
endproc
```

Llamamos *d* a la cantidad máxima de dígitos de los elementos y suponemos que el dígito 1 es el menos significativo de cada valor.

Complejidad : O(d × ordenar A por un dígito).

Con Bucket Sort (sin ordenar los buckets): O(n · d).

Radix Sort (Generalizado)

- Radix Sort puede usarse también para ordenar **cadenas de caracteres** (la primera letra es la más significativa).
- El mismo esquema sirve para ordenar **tuplas**, donde el orden que se quiera dar depende principalmente de un “dígito” (i.e., un campo de la tupla) y en el caso de empate se tengan que revisar los otros.
- La idea en el fondo es la misma: usar un algoritmo estable e ir ordenado por “dígito” (del menos al más significativo).

Radix Sort (Generalizado)

```
proc RadixSort(A: Array<T>, d: int)

    for (i=0, i <= d, i+1)
        // Del menos significativo al más significativo.
        Ordenar el arreglo A según el dígito i,
        de forma estable
    endfor
endproc
```

¡Ejercicios!

Ejercicio 18. Dado un arreglo de pares `<estudiante, nota>`, queremos devolver un arreglo de pares `<estudiante, promedio>`, ordenado en forma descendente por promedio y, en caso de empate, por número de libreta.

Los estudiantes se representan con un número de libreta (puede considerar que es un número de a lo sumo 10 dígitos), y la nota es un número entre 1 y 100. La cantidad de estudiantes se considera no acotada.

Se pide dar un algoritmo que resuelva el problema con complejidad $O(n + m \log m)$, donde n es la cantidad total de notas (es decir, el tamaño del arreglo de entrada), y m es la cantidad total de estudiantes.

Ejemplo:

```
Entrada: <12395, 72>, <45615, 81>, <12395, 94>, <45615, 100>, <78920, 81>,
         <12395, 90>, <45615, 71>, <78920, 50>
Salida: <12395, 85.3333>, <45615, 84>, <78920, 65.5>
```

a) Describa cada etapa del algoritmo con sus palabras, y justifique por qué cumple con las complejidades pedidas.

b) Escriba el algoritmo en pseudocódigo. Puede utilizar (sin reescribir) todos los algoritmos y estructuras de datos vistos en clase.

Idea:

1) Como sabemos que las libretas tienen **a lo sumo** 10 dígitos podemos usar Radix Sort para ordenar el array por libreta de manera ascendente **O(10 · n) = O(n)**
2) Recorremos el array ordenado por libreta y calculamos la *sum_notas* y *cant_notas* de cada alumno obteniendo un **array(< Estudiante, promedio >) O(n)**
3) Ordenamos con algoritmo **estable**, manteniendo el orden anterior, MergeSort por promedio de manera descendiente **O(m log m)**
4) Devolvemos el array ordenado por ambos criterios **O(1)**

Complejidad final **= O(n) + O(m log m)**

¡¡Intervalo!!



Ejercicio de Parcial 🐕

b) Se está por realizar un gran censo de perros en la Ciudad de Buenos Aires. La información recaudada llegará en forma de un arreglo de tuplas `<Nombre, Raza, Edad, Peso>`. Las razas son `strings` de largo acotado L, pero no sabemos cuántas razas pueden existir, aunque sí sabemos que entre ellas está la mejor raza de todas: raza Perro. Las edades serán enteros positivos entre 1 y 20 y los pesos serán de tipo `float`.

Proponga un algoritmo que organice la información en dos arreglos:

- Uno con los perros raza Perro ordenados por edad y, en caso de empate, por peso, ambos en forma ascendente
- Otro arreglo con la información de las razas y la cantidad de perros de cada una en forma de tuplas `<Raza, int>`, ordenadas en forma decreciente por la cantidad y, en caso de empate, alfabéticamente por la raza

La complejidad temporal del algoritmo debe ser $O(n + p.log(p))$, donde n es el total de perros y p el total de perros raza Perro.

Ejemplo

Si recibimos:

```
[ <Nulus, Perro, 4, 15.3>, <Joe, Cocker, 14, 8.0>, <Lady, Caniche, 10, 6.1>, <Dobby, Perro, 7, 18.3>
<Res, Dogo, 3, 50.0>, <Haru, Perro, 7, 30.0>, <Clementina, Perro, 14, 10.0>, <Choqui, Caniche, 16, 5.0>]
```

Deberíamos devolver:

```
P: [ <Nulus, 4, 15.3>, <Dobby, 7, 18.3>, <Haru, 7, 30.0>, <Clementina, 14, 10.0>]

R: [ <Caniche, 2>, <Cocker, 1>, <Dogo, 1>]
```

Idea

1) Separamos los que son raza perro de los que no: **O(n)**
- Sabemos que las razas son strings de **largo acotado L**, por lo que podemos usar un dict Trie (Raza, int) para guardar las razas con sus cantidades.
- Los que son de raza perro los guardamos en un array(< Nombre, Edad, Peso >)

2) Ordenamos:
- Los Raza Perro con **Merge Sort**, primero por peso y luego por edad de manera asc. (mantenemos el orden del primer ordenamiento ya que Merge es **estable**) **O(p log p)**
- Los de Raza los pasamos a un array de tuplas y ordenamos con **Radix Sort** para ordenar alfabéticamente (pues, largo acotado L) y luego por cantidad con **Bucket Sort** (sabemos que son a lo sumo n perros y es estable) **O(n · L) + O(n+n) = O(n)**

3) Devolvemos ambos arrays ordenados **O(1)**

Complejidad final = **O(n) + O(p log p)**

Ejercicio 12. Se desea ordenar los datos generados por un sensor industrial que monitorea la presencia de una sustancia en un proceso químico. Cada una de estas mediciones es un número entero positivo. Dada la naturaleza del proceso se sabe que, dada una secuencia de n mediciones, a lo sumo $\lfloor \sqrt{n} \rfloor$ valores están fuera del rango $[20, 40]$.

Proponer un algoritmo $O(n)$ que permita ordenar ascendentemente una secuencia de mediciones y justificar la complejidad del algoritmo propuesto.

Idea

1) Separo los que caen dentro del rango [20-40] de los que no: los separamos en dos listas: *EnRango* y *FueraDeRango*. **O(n)**
2) Tenemos que ver que ordenamiento hacemos en cada una:

Obs! *Los algoritmos de ordenamiento que vimos toman como parámetro un array, no listas. Vamos a tener que pasar las listas a arrays.*

- Sabemos que a lo sumo hay **√n** elementos que están **fuera de rango**, es decir, que en el peor caso pueden ser **n** la cantidad de números **dentro del rango**.
- Como la complejidad nos pide O(n) tenemos que tener en cuenta que para los elementos que caen dentro del rango deberíamos aplicar un algoritmo lineal.

Idea

Teniendo esto en cuenta:

- Para los elementos dentro del rango usamos **Counting Sort** ya que sabemos que son **naturales** y están **acotados** **$O(n+40) = O(n)$**
- Para los que están fuera del rango podemos usar **Insertion Sort**, entre otros. **$O(\sqrt{n^2}) = O(n)$**

3) Crear el array final concatenando los menores a 20 primero, luego los que están dentro del rango y por último los mayores a 40, y devolvemos ese array **$O(n)$**

Complejidad final **$= O(n)$**